

OCRRipple

Measuring how OCR errors ripple through document-understanding pipelines

Technical Report

Project: OCRRipple

Repository: github.com/Sarvarbek-Erniyazov/OCRRipple

Author: Sarvarbek Erniyazov

InnoCORE Global Postdoctoral Program - candidate portfolio project

1. Motivation

Document-understanding systems (RAG, QA, search) are commonly evaluated by the OCR engine's own accuracy metric — character error rate (CER) or word error rate (WER). This report tests whether that metric is the right one to optimize, by measuring CER *and* two downstream tasks side by side, across three OCR paradigms: classical (Tesseract), production (PaddleOCR), and vision-language-model-based (DeepSeek-OCR). A fourth engine, Baidu Unlimited-OCR, was run but its results are excluded — see Section 8.

2. Method

2.1 Datasets

Controlled synthetic set. Three arXiv papers were rendered to page images at 200 DPI and degraded at four controlled levels (clean → light → medium → heavy) using OpenCV: Gaussian blur, sensor noise, uneven illumination, JPEG compression artifacts, and slight rotation (seeded, reproducible). Ground truth is the exact PDF text layer — no OCR or annotation error is possible. 172 page images across 3 papers × 4 levels.

Real-world set. A 50-document, 300-question subset of MP-DocVQA (Imms-lab mirror), preserving gold answer text and gold answer-page index per question. 350 total pages.

Layout set. 500 pages sampled from the DocLayNet-v1.1 test shard (11 classes: Text, Title, Table, Picture, Formula, List-item, Caption, Footnote, Page-header, Page-footer, Section-header), exported to COCO format for Roboflow.

2.2 OCR engines

Engine	Version / model	Compute	Notes
Tesseract	5.5.0	local CPU	pytesseract wrapper
PaddleOCR	PP-OCRv6 (medium)	local CPU	enable_mkldnn=False required (oneDNN/PIR incompatibility)
DeepSeek-OCR	HF deepseek-ai/DeepSeek-OCR	Kaggle T4	bfloat16, eager attention; float16 caused dtype mismatch
Baidu Unlimited-OCR	HF baidu/Unlimited-OCR	Kaggle T4	shares DeepSeekV2 backbone; results excluded, see Sec. 8

2.3 Layout detection

RF-DETR (Small) was trained on the 500-page DocLayNet subset with no data augmentation (document layout has strong positional priors — header always top, footer always bottom — that flip/rotate augmentation would corrupt). Result: **mAP@50 73.8%**, Precision 76.9%, Recall 71.8%, F1 74.3%. Deployed as a Roboflow-hosted inference API.

2.4 Downstream tasks

Stage 1 — Retrieval + Answer Presence (MP-DocVQA). For each question, all pages of its source document are embedded with all-MiniLM-L6-v2 and ranked by cosine similarity to the question. We report Hit@3 (is the gold answer page retrieved in the top 3?) and an ANLS-style Answer-Presence score (best Levenshtein-normalized similarity between the gold answer and any 60-character window of the top-3 evidence text).

Stage 2 — Fact Recall (arXiv synthetic set). Email addresses are extracted from ground truth via regex and checked for exact survival in each engine's OCR output, per degradation level. This isolates a single, binary-scoreable fact from the confound of text-ordering or paraphrase.

3. Results

3.1 CER by degradation level (normalized)

Level	Tesseract	DeepSeek-OCR	PaddleOCR
clean	7.3%	14.0%	47.3%
light	7.6%	15.2%	47.4%
medium	8.4%	14.1%	47.6%
heavy	12.4%	13.6%	48.1%

Tesseract's CER rises ~70% relative from clean to heavy. DeepSeek-OCR's is flat, even dipping slightly. PaddleOCR's is flat but at a much higher baseline (see 3.3).

3.2 Downstream task results

Engine	Hit@3	Answer ANLS	Email recall (clean)	Email recall (heavy)
Tesseract	0.833	0.712	1.00	0.75
DeepSeek-OCR	0.797	0.816	1.00	1.00
PaddleOCR	0.827	0.848	1.00	1.00

Robustness check: restricting to the harder subset of MP-DocVQA questions (>3 candidate pages, ~65% of the set) preserves the ranking — Hit@3 of 0.738 (Tesseract), 0.687 (DeepSeek-OCR), 0.733 (PaddleOCR) — ruling out a trivial “few pages, easy retrieval” artifact.

3.3 Why PaddleOCR's CER is misleading

Manual diff of attention_p000.png shows PaddleOCR recognizes every token correctly (Ashish Vaswani*, avaswani@google.com, etc.) but groups them by visual column rather than reading order: all four author names, then all four affiliations, then all four emails — instead of name→affiliation→email per author. This is a **reading-order** failure, not a **recognition** failure, but a token-level CER metric penalizes both identically. A vertical sidebar (the arXiv identifier) is also misread as horizontal text (“3rXi 1 vvv 023” for “arXiv:1706.03762v7”).

This explains the central paradox: word-level, order-insensitive downstream tasks (embedding retrieval, exact-substring fact recall) are largely immune to reading-order noise, while CER is maximally sensitive to it.

4. Central finding

Raw CER does not predict downstream answer quality. PaddleOCR has the worst CER (47–48%, driven entirely by reading order, not recognition) and the best Answer ANLS and email recall. Tesseract has the best CER and the worst Answer ANLS, plus a 25-point email-recall drop under heavy degradation that neither other engine shows. The type of error (ordering vs. recognition vs. omission) matters more than its raw quantity for tasks that consume OCR output downstream.

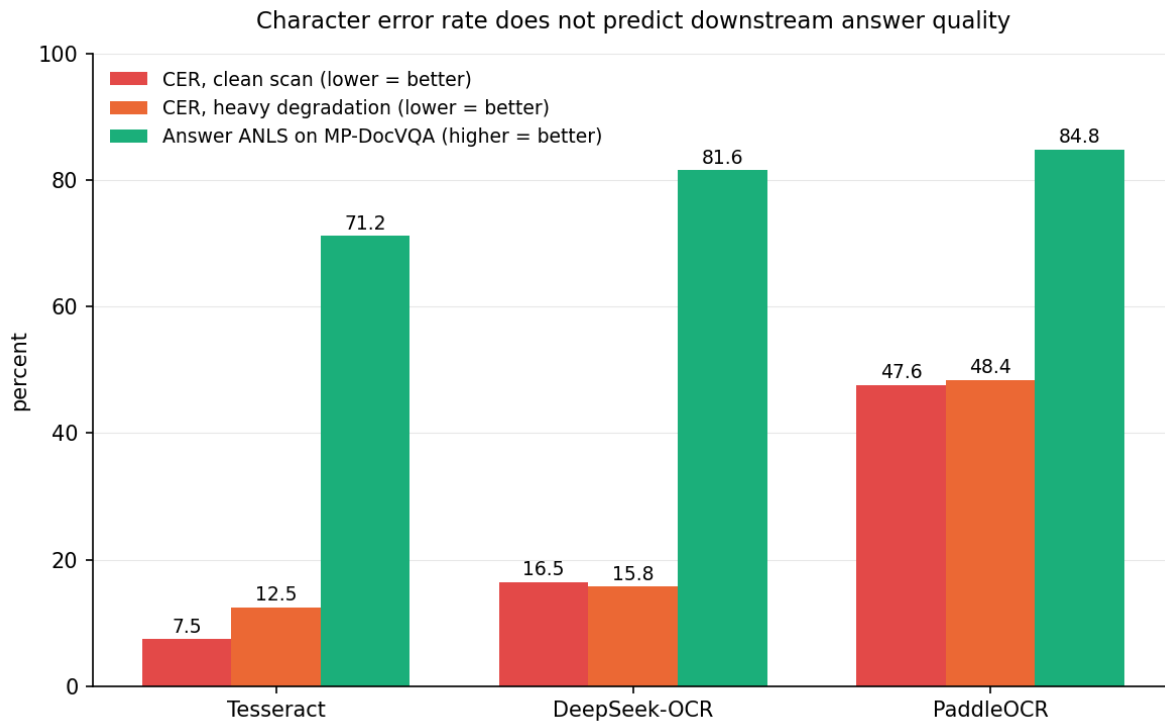


Figure 1. Character error rate (clean and heavy degradation) vs. downstream Answer ANLS, per engine.

5. Layout-aware analysis: does Table content hurt more?

Using the deployed Roboflow layout model (Section 2.3), we tagged each question's gold answer page with whether it contains a Table region (98 of 300 questions do), then compared Answer ANLS between Table-pages and non-Table-pages, per engine.

Engine	ANLS, no Table	ANLS, has Table	Relative drop
Tesseract	0.778	0.577	-25.9%
DeepSeek-OCR	0.827	0.793	-4.2%
PaddleOCR	0.854	0.834	-2.3%

Tesseract's answer quality drops sharply on pages containing tables, while DeepSeek-OCR and PaddleOCR remain comparatively flat. This localizes the “CER doesn't predict downstream quality” finding

(Section 4) to a specific, actionable cause: classical OCR's difficulty with tabular structure — not just the multi-column reading-order issue described in Section 3.3 (which affected author blocks), but a broader pattern across real-world tabular content in MP-DocVQA. Practical implication: for table-heavy document collections, engine choice matters more than aggregate CER would suggest.

6. Limitations

- Sample size is small (3 papers / 172 synthetic pages, 50 documents / 300 questions) — appropriate for a one-week exploratory study, not a large-scale benchmark.
- The retrieval encoder (all-MiniLM-L6-v2) is fixed across engines; a larger or fine-tuned retriever might behave differently with respect to reading-order noise.
- CER normalization (stripping markdown symbols) is coarse and was not independently validated against a human-annotated gold normalization.

7. Reproducibility notes

Getting VLM OCR engines running on Kaggle required resolving, in order: a missing addict dependency, a transformers API mismatch (LlamaFlashAttention2 removed), an `is_torch_fx_available` import error after pinning transformers too far back, a `float16/float32 masked_scatter_ dtype` mismatch, and an `nVRTC` CUDA-kernel compilation failure inside a fused `quick_gelu` op. All were resolved without modifying model weights or logic — only environment/dtype/JIT configuration.

8. Unlimited-OCR: run completed, results excluded

Baidu's Unlimited-OCR (GitHub: [baidu/Unlimited-OCR](#), HuggingFace: [baidu/Unlimited-OCR](#)) is a 3B-parameter Mixture-of-Experts model (500M active), using Reference Sliding Window Attention (R-SWA) — attending jointly to the source document, nearby context, and next tokens while progressively discarding stale context — to keep KV-cache size constant. Baidu reports SOTA on OmniDocBench v1.5/v1.6 and processing 40+ pages in one forward pass within a 32K context window without losing context or slowing generation.

Unlimited-OCR ran to completion on all 522 pages (Kaggle T4, `bfloat16`, JIT GPU fusion disabled to work around an `nVRTC` kernel-compilation issue in the vision encoder). However, **we do not report its downstream metrics as valid**, based on the following output-integrity audit:

Engine	Empty outputs (of 522)	Rate
Tesseract	5	1.0%
DeepSeek-OCR	1	0.2%
PaddleOCR	0	0.0%
Unlimited-OCR	279	53.4%

The other three engines fall within normal noise (isolated corrupt source images). Unlimited-OCR's 53.4% is not model failure: `timing.json` shows these pages consumed normal-to-long inference time (e.g. 22.9s for `attention_p002.png`) before yielding an empty *post-processed* result. The cause is a bug in our own tag-stripping regex (`re.sub(r'<\|det\|>[^\<]*<\|/det\|>', '', text)`), which assumes each `<|det|>` tag closes before the next one opens. On densely-tagged pages this assumption breaks and the regex silently

consumes real generated text along with the tags. Raw (untagged) output was not persisted, so this cannot be corrected retroactively — only by re-running inference with a corrected regex, which was out of scope for this one-week project.

Relation to Baidu's claims: because our numbers reflect a bug in our pipeline rather than the model, they neither confirm nor refute Unlimited-OCR's published claims. The closest available evidence is DeepSeek-OCR, which shares the same DeepSeekV2 backbone family: it showed flat CER and 100% email recall across all degradation levels, versus Tesseract's 25% recall loss at the heaviest level — consistent with, though not a full validation of, R-SWA's stated robustness goal.